

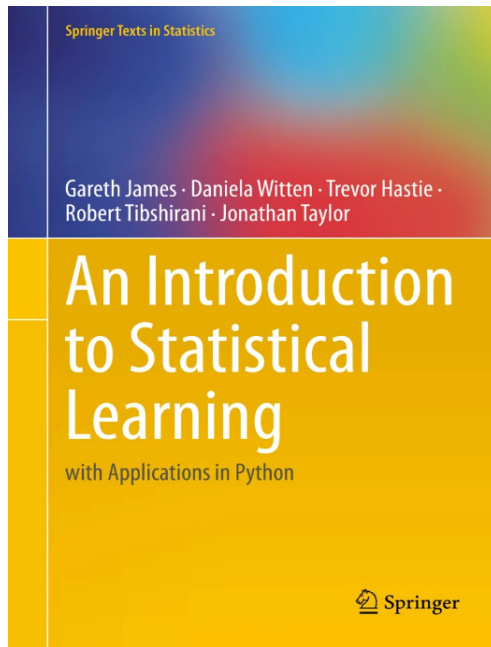
Machine Learning with Applications in Python

Teshome K. Deressa^{a,b,c}

^aGuest Research Fellow
Centre for Environmental Sciences
[Hasselt University, Belgium](#)

^bPostdoctoral Researcher
Centre for Neuroscience & Cell Biology
[University of Coimbra, Portugal](#)

^cData Analyst – Freelancer
Framework for **O**pen & **R**e reproducible **R**esearch **T**raining – **FORRT**
[Remote, Germany](#)



Disclaimer : In addition to this book, I have used free online resources while preparing this material. Also, kindly know that the topics I presented are introductory!



Outline (1)

① Introduction – definition of key terms

② Types of ML

③ First things first

④ Prediction

- Linear regression (OLS)
- Ridge regression (ℓ_2 regularization) ⚠
- Lasso regression (ℓ_1 regularization) ⚠
- ElasticNet (combined ℓ_1 and ℓ_2) ⚠

⑤ Classification

- Logistic regression
- k -Nearest Neighbors (k -NN)
- Support Vector Machines (SVM)
- Tree-based methods (bagging, random forest, boosting, gradient boosting, XGBoost) ⚠



⑥ Unsupervised learning: Clustering

- k -Means clustering
- Hierarchical clustering ⚠
- Density-Based Spatial Clustering of Applications with Noise (DBSCAN) ⚠
- Topic modeling ⚠
- Principal component analysis ⚠

⑦ Take-home message



Introduction (1)



Many words, many concepts



Artificial Intelligence (AI)

- Branch of computer science dedicated to designing systems capable of performing tasks that would normally require human intelligence. These tasks include reasoning, learning, perception, understanding natural language, and decision-making in uncertain environments
- Any technique that enables computers to mimic human behavior

Generative AI (GenAI)

- Class of AI models capable of **creating new content**—such as text, images, audio, or code—by learning the underlying patterns of existing data. These models do not merely classify or predict; they generate data that resembles the training data, often using architectures such as Large Language Models (LLMs)
- ChatGPT of **OpenAI**, Gemini of **Google**, Grok of **xAI**, etc.



Introduction (3)

Natural Language Processing (NLP)

- Subset of AI that focuses on the automatic analysis, understanding, and generation of **text** written in natural language. It enables machines to process and extract meaning from human language in textual form, using computational and statistical techniques

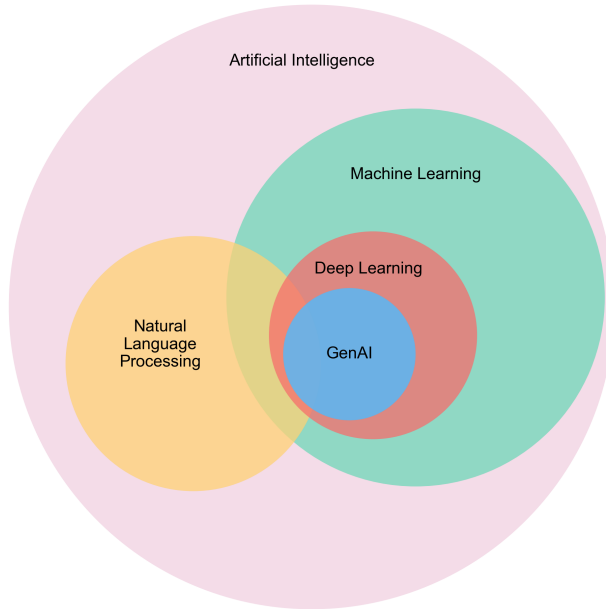
Machine Learning (ML)

- Subset of AI that focuses on developing algorithms and statistical models that allow computers to perform specific tasks without being explicitly programmed. Instead, these systems learn patterns and rules from data and improve their performance through experience
- Ability to learn without explicitly being programmed

Deep Learning (DL)

- Subset of ML
- Extract patterns from data using neural networks with many layers





Introduction (5)

1950's	 Artificial intelligence (AI) <i>Human intelligence exhibited by machines</i>		
	1980's	 Machine learning <i>AI systems that learn from historical data</i>	
		2010's	 Deep learning <i>Machine learning models that mimic human brain function</i>
			 Generative AI (Gen AI) <i>Deep learning models (foundation models) that create original content</i>

Source: [IBM 2025](#)



Based on the structure of the data and the type of feedback available, tasks are categorized into different **learning modes** in ML

① Supervised learning

- algorithm is trained on **labelled** data → each input x_i is associated with an output y_i
- goal is to teach a model that maps inputs to outputs so that it can predict labels for new data
- used for prediction or classification

② Unsupervised learning

- data is **unlabelled**
- goal is to discover hidden patterns in the input data x_i , such as clusters



③ Semi-supervised learning (*aka hybrid*) ⚠

- combines a small amount of labeled data with a large amount of unlabeled data
- algorithm leverages both to improve learning accuracy

④ Reinforcement learning ⚠

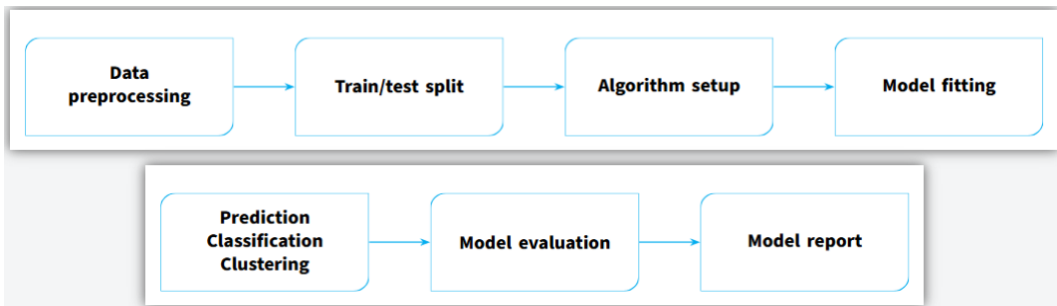
- an agent learns to make decisions by interacting with an environment and receiving feedback in the form of **rewards or penalties**



First things first (1)

Wait! 🤔

- What's the business problem?
- Take time in identifying and understanding the business problem



General steps to build ML models



First things first (2)

- We can build ML models using modules and libraries implemented in Python
 - numpy
 - pandas
 - matplotlib
 - seaborn
 - scipy
 - **sckit-learn** – focus on this! 

Why sckit-learn?

- Covers a wide range of ML tasks
- Easy to implement
- Community and documentation



Supervised ML algorithms

Quick recap – SLR (1)

- Predicts a continuous dependent variable based on input feature
- Model: $y_i = \beta_0 + \beta_1 x_1 + \epsilon_i$

Applications

- Estimating house prices based on property features
- Predicting the future sales of a product
- Predicting crop yields
- $\vdots$

Example

- Consider `fuel_consump.csv` dataset
- Explore the dataset
- Objective: check whether `engine_size` predict `co2_emmission`
- Let's try! 😊



Quick recap – SLR (2)

```
1
2 import numpy as np
3 import pandas as pd
4 import matplotlib.pyplot as plt
5 import seaborn as sn
6
7 df = pd.read_csv('C:/Users/Boss/OneDrive - Universidade de Coimbra/Ambiente de Trabalho
8 df.head()
9 df.describe()
10
11 plt.scatter(df['engine_size'], df['co2_emission'], color='blue')
12 plt.xlabel('Engine Size')
13 plt.ylabel('CO2 Emissions')
14 plt.show()
```



Quick recap – SLR (3)

- Identify feature (X) and target variable (y)

```
1 X = df[['engine_size']]
2 y = df['co2_emission']
```

- Split the dataset into train and test

```
1 from sklearn.model_selection import train_test_split
2 X_train, X_test, y_train, y_test = train_test_split(X, y,
3                                                    test_size=0.2, random_state=599)
```

- Fit the model

```
1 from sklearn.linear_model import LinearRegression
2 model = LinearRegression()      # create the model
3 model.fit(X_train, y_train)     # fit the model
4 y_pred = model.predict(X_test)  # make predictions on the test data
5
6 print(model.coef_, model.intercept_)
```



- Evaluate the model
 - calculate some metrics to evaluate the model's performance

```
1 from sklearn.metrics import mean_squared_error, r2_score
2 import math
3 mse = mean_squared_error(y_test, y_pred)
4 rmse = math.sqrt(mse) # root mean square error
5 r2 = r2_score(y_test, y_pred)
6
7 print(mse, rmse, r2)
```



Multiple linear regression (1)

- $co2_emission = f(engine_size, consum_comb, cylinders)?$

```
1 X = df[['engine_size', 'cylinders', 'consump_comb']]
2 y = df['co2_emission']
3
4 from sklearn.model_selection import train_test_split
5 from sklearn.linear_model import LinearRegression
6 from sklearn.metrics import mean_squared_error, r2_score
7
8 X_train, X_test, y_train, y_test = train_test_split(X, y,
9                                                    test_size=0.2, random_state=55)
10 model = LinearRegression()
11 model.fit(X_train, y_train)
12 print(model.coef_, model.intercept_)
13
14 y_pred = model.predict(X_test)
```



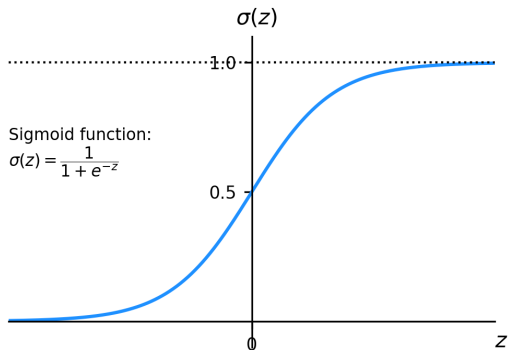
Multiple linear regression (2)

```
1  mse = mean_squared_error(y_test, y_pred)
2  rmse = math.sqrt(mse)
3  r2 = r2_score(y_test, y_pred)
4  residuals = y_test - y_pred
5  print(mse, rmse, r2)
6
7  # 1. Linearity: Is that OK?
8  plt.scatter(y_test, y_pred)
9  plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--')
10 plt.xlabel('Actual Values')
11 plt.ylabel('Predicted Values')
12 plt.title('Actual vs Predicted Values')
13 plt.show()
14
15 # 2. Residuals plot: Is that OK?
16 plt.scatter(y_pred, residuals)
17 plt.axhline(y=0, color='r', linestyle='--')
18 plt.xlabel('Predicted Values')
19 plt.ylabel('Residuals')
20 plt.title('Residuals vs Predicted Values')
21 plt.show()
```



Logistic regression (1)

- It's a variation of LR when the dependent variable is **categorical**
- Predicts the probability that a given input belongs to a specific class (e.g., Yes/No)
- Use **sigmoid function** to convert the input values into a probability between 0 & 1



Decision rule

- $z = \beta_0 + \beta_1 x_1 + \dots + \beta_p x_p$
- If $\sigma(z) \geq 0.5 \rightarrow$ predict class 1 (**Yes**)
- If $\sigma(z) < 0.5 \rightarrow$ predict class 0 (**No**)



Example

- Consider the telecom dataset: `churn.csv`
- The dataset includes information about:
 - customers who left within the last month
 - services that each customer has signed up for
 - customer account information
 - demographic info about customers
- Objective: predicting customer churn or attrition (churn *vs* not churn)
- Let's try! 😊



Logistic regression (3)

```
1 import numpy as np
2 import pandas as pd
3 from sklearn.model_selection import train_test_split
4 from sklearn.preprocessing import StandardScaler
5 from sklearn.linear_model import LogisticRegression
6 from sklearn.metrics import classification_report, confusion_matrix, ConfusionMatrixDisplay
7
8 # Load the data & explore briefly
9 df = pd.read_csv('C:/Users/Boss/OneDrive - Universidade de Coimbra/Ambiente de Trabalho')
10 df.head()
11 df['churn'].value_counts()
12
13 # Split features/target
14 X = df[['tenure', 'age', 'address', 'income', 'ed', 'employ', 'equip']]
15 y = df['churn'] # 0 = not churn, 1 = churn
16
17 X_train, X_test, y_train, y_test = train_test_split( X, y,
18                                                         test_size=0.2, random_state=44)
```



Logistic regression (4)

```
1  # Feature scaling (helps with logistic regression optimization)
2  scaler = StandardScaler()
3  X_train_scaled = scaler.fit_transform(X_train) # calculate mean & std and scale
4  X_test_scaled = scaler.transform(X_test) # only scaling using mean & std from training
5
6  # Fit logistic regression model
7  model = LogisticRegression(max_iter=1000)
8  model.fit(X_train_scaled, y_train)
9
10 # Predictions
11 y_pred = model.predict(X_test_scaled)
```



Logistic regression (5)

```
1  # Evaluation
2  cm = confusion_matrix(y_test, y_pred, labels=[1,0]) # standardized
3
4  # Plot with clear labels
5  cmp = ConfusionMatrixDisplay(confusion_matrix=cm,display_labels=['Churn','Not churn'])
6  cmp.plot(cmap='Blues')
7  plt.gca().xaxis.tick_top() # move tick labels
8  plt.gca().xaxis.set_label_position('top') # move axis label
9  plt.xlabel('Predicted label') # reset label text
10 plt.ylabel('True label') # keep true labels on side
11 plt.show()
12
13 print('\nClassification report:\n', classification_report(y_test, y_pred,
14                                                         target_names=['Not churn', 'Churn']))
15
16 # Check model coefficients
17 coeffs = pd.DataFrame({
18     'Feature': X.columns,
19     'Coefficient': model.coef_[0]
20 }).sort_values(by='Coefficient', ascending=False)
21
22 print('\nTop predictors of churn:\n', coeffs.head(10))
```



Confusion matrix – extra (1)

- Table used to evaluate the performance of a classification model
- Compares the predicted labels with the actual (true) labels

	Predicted Positive	Predicted Negative
Actual Positive	True Positive (TP)	False Negative (FN)
Actual Negative	False Positive (FP)	True Negative (TN)

Brainstorming 🤔

- Type I error – α ?
- Type II error – β ?



Classification report metrics – extra (2)

- ① **Accuracy:** Proportion of correctly predicted observations

$$\text{Accuracy} = \frac{TP+TN}{TP+TN+FP+FN}$$

- ② **Precision:** Of all predicted positives, how many are actually positive?

$$\text{Precision} = \frac{TP}{TP+FP}$$

- ③ **Recall:** Of all actual positives, how many were correctly predicted?

$$\text{Recall} = \frac{TP}{TP+FN}$$

- ④ **F1 Score:** Harmonic mean of precision and recall

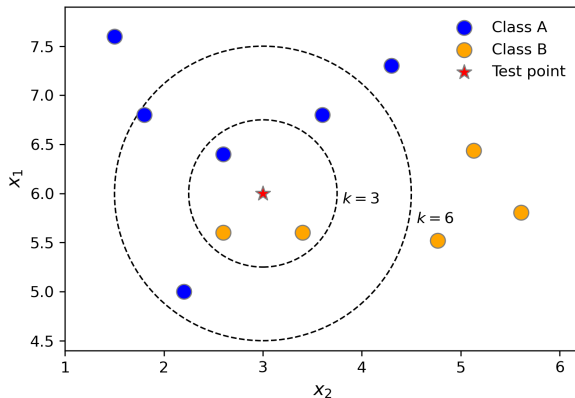
$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

- ⑤ See also **ROC curve** and **AUC**



k -Nearest Neighbors – k -NN (1)

- Assigns the new data point to the class that is most common among its k -nearest neighbours → **majority voting**
- Based on a distance metric → Euclidean or Manhattan
- Feature scaling is crucial
- Choosing optimal k is not easy but very important



Example

- Consider `tele_cust.csv` dataset
- Target variable $\rightarrow$ `custcat`
 - 1- Basic service, 2- eService, 3- Plus service, 4- Total service
- Explore the dataset
- Objective: build a k -NN classifier to predict the class of an unknown customer
- Let's try! 😊

```
1 import pandas as pd
2 import numpy as np
3 from sklearn.model_selection import train_test_split
4 from sklearn.preprocessing import StandardScaler
5 from sklearn.neighbors import KNeighborsClassifier
6 from sklearn.metrics import classification_report, confusion_matrix, ConfusionMatrixDisplay
```



```
1  # Load the data & inspect
2  df = pd.read_csv('C:/Users/Boss/OneDrive - Universidade de Coimbra/Ambiente de Trabalho
3  df.head()
4  df.dtypes
5  df['custcat'].value_counts()
6
7  # Split features & target
8  X = df[['region', 'tenure', 'age', 'marital', 'address', 'income', 'ed',
9          'employ', 'retire', 'gender', 'reside']]
10 y = df['custcat']
11
12 # Train/test split
13 X_train, X_test, y_train, y_test=train_test_split( X,y,test_size=0.2,random_state=4)
14
15 # Standardize features
16 scaler = StandardScaler()
17 X_train_scaled = scaler.fit_transform(X_train)
18 X_test_scaled = scaler.transform(X_test)
19
20 # Fit KNN model (start with k=4)
21 model = KNeighborsClassifier(n_neighbors=4)
22 model.fit(X_train_scaled, y_train)
```



```
1  # Predictions
2  y_pred = model.predict(X_test_scaled)
3
4  # Evaluation
5  cm = confusion_matrix(y_test, y_pred, labels=[1,2,3,4])
6
7  # Plot with clear labels
8  cmp = ConfusionMatrixDisplay(confusion_matrix=cm,
9                               display_labels=['Base', 'eService', 'Plus', 'Total'])
10 cmp.plot(cmap='Blues')
11 # Move the x-axis labels to the top
12 plt.gca().xaxis.tick_top()           # move tick labels
13 plt.gca().xaxis.set_label_position('top') # move axis label
14 plt.xlabel('Predicted label')         # reset label text
15 plt.ylabel('True label')              # keep true labels on side
16 plt.show()
17
18 print('\nClassification report:\n', classification_report(y_test, y_pred,
19                                                             target_names=['Base', 'eService', 'Plus', 'Total']))
```

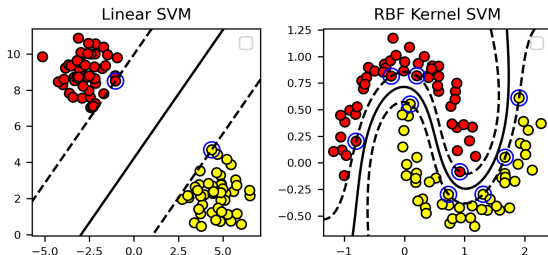


Support Vector Machine – SVM (1)

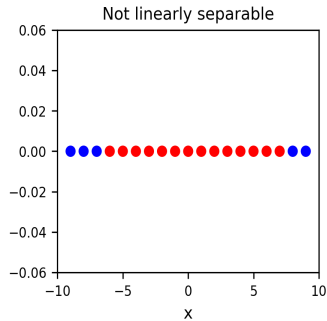
- Classify the cases by finding a **separator**
- Handle high-dimensional data and non-linearly separable datasets

How we find a separator?

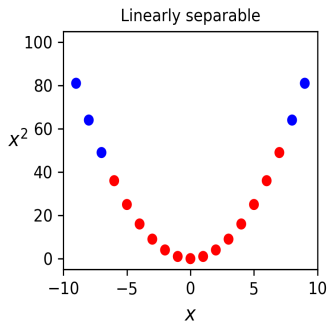
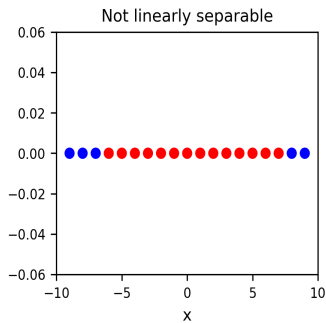
- **Mapping data to a higher-dimensional space** → Kernelling
- Kernelling (all have their own pros and cons):
 - linear
 - polynomial
 - sigmoid
 - radial basis function (RBF) – the default in Python



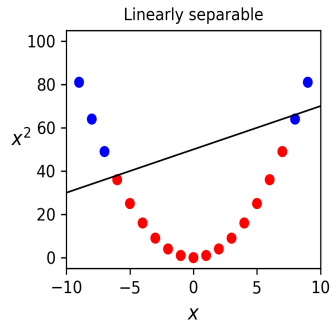
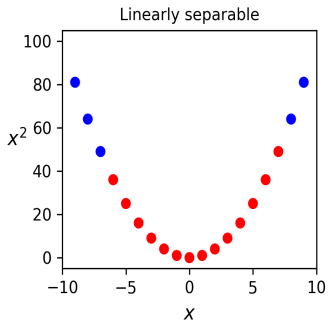
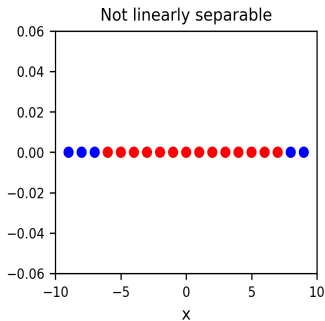
- $\psi(x) = [x, x^2] \rightarrow$ polynomial mapping



- $\psi(x) = [x, x^2] \rightarrow$ polynomial mapping



- $\psi(x) = [x, x^2] \rightarrow$ polynomial mapping



Applications

- Email classification (spam *vs.* non-spam)
- Fraud detection (fraud *vs.* non-fraud)
- Human cell classification (benign *vs.* malignant)
- $\vdots$

Advantages

- Works well for high-dimensional data
- Memory efficient → rely on support vectors

Disadvantages

- Prone to **overfitting**
- Small dataset



Example

- Consider the `cell_samples.csv` dataset
- The dataset consists human cell sample records

Variable name	Description
id	Patient identifier
clump	Clump thickness
unif_size	Uniformity of cell size
unif_shape	Uniformity of cell shape
marg_adh	Marginal adhesion
sing_epi_size	Single epithelial cell size
bare_nuc	Bare nuclei
bland_chrom	Bland chromatin
norm_nucl	Normal nucleoli
mit	Mitoses
class	Benign (=2) or malignant (=4)

- Objective: classify new sample cells as benign or malignant
- Let's do it! 😊

```
1 import pandas as pd
2 import numpy as np
3 from sklearn import preprocessing
4 from sklearn.model_selection import train_test_split
5 from sklearn import svm
6 from sklearn.metrics import classification_report, confusion_matrix, ConfusionM
7
8 df = pd.read_csv('C:/Users/Boss/OneDrive - Universidade de Coimbra/Ambiente d
9 df.head()
10 df.shape
11 df.dtypes
12
13 # It looks like the bare_nuc includes some values that are not numerical
14 # Convert all columns to numeric (some entries may be '?')
15 df = df.apply(pd.to_numeric, errors='coerce')
16
17 # remove rows with missing values
18 df = df.dropna()
```



```
1  # Target/features
2  X = df[['clump', 'unif_size', 'unif_shape', 'marg_adh',
3  'sing_epi_size', 'bare_nuc', 'bland_chrom', 'norm_nucl', 'mit']]
4  y = df['class']
5
6  # Train/test dataset
7  X_train, X_test, y_train, y_test = train_test_split( X, y,
8  test_size=0.2, random_state=77)
9
10 # Fit the model
11 model = svm.SVC(kernel='rbf') # RBF is the default
12 model.fit(X_train, y_train)
13 #model = svm.SVC(kernel='linear')
```

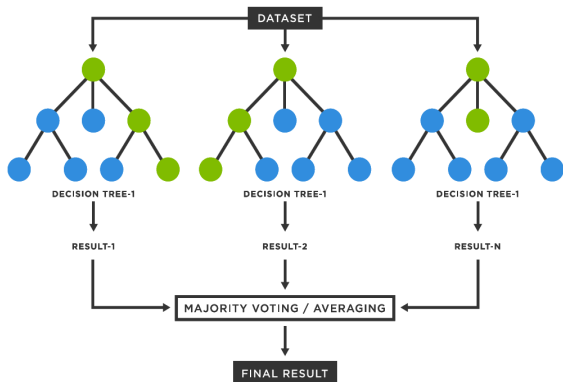


```
1  # Make predictions
2  y_pred = model.predict(X_test)
3
4  # Evaluation
5  cm = confusion_matrix(y_test, y_pred, labels=[4,2])
6
7  # Plot with clear labels
8  disp = ConfusionMatrixDisplay(confusion_matrix=cm,
9                                display_labels=['Malignant','Benign'])
10 disp.plot(cmap='Blues')
11 # Move the x-axis labels to the top
12 plt.gca().xaxis.tick_top()           # move tick labels
13 plt.gca().xaxis.set_label_position('top') # move axis label
14 plt.xlabel('Predicted label')        # reset label text
15 plt.ylabel('True label')             # keep true labels on side
16 plt.show()
17
18 print('\nClassification report:\n',classification_report(y_test,y_pred,
19                                                            target_names=['Malignant','Benign']))
```



Random Forest (1)

- Ensemble algorithm that combines many decision trees to improve predictions or classification
- Each tree is built using random samples of the data & a random subset of features
- Random sampling is with replacement → bootstrapping
- Models are built simultaneously – bagging



Note!

- Classification → majority voting
- Prediction → averaging
- Aggregation → reduce **overfitting** & improve accuracy



Example

- Reconsider the human cell dataset: `cell_samples.csv`
- Objective: classify new sample cells as benign or malignant
- Let's do it! 😊

```
1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4 from sklearn import preprocessing
5 from sklearn.model_selection import train_test_split
6 from sklearn.ensemble import RandomForestClassifier
7 from sklearn.metrics import classification_report, confusion_matrix, ConfusionM
8
9 df = pd.read_csv('C:/Users/Boss/OneDrive - Universidade de Coimbra/Ambiente d
10 df.head()
11 df.shape
12 df.dtypes
```



Random Forest (3)

```
1  # Convert all columns to numeric (some entries may be '?')
2  df = df.apply(pd.to_numeric, errors='coerce')
3
4  # remove rows with missing values
5  df = df.dropna()
6
7  # Target/features
8  X = df[['clump', 'unif_size', 'unif_shape', 'marg_adh',
9  'sing_epi_size', 'bare_nuc', 'bland_chrom', 'norm_nucl', 'mit']]
10 y = df['class']
11
12 # Train/test dataset
13 X_train, X_test, y_train, y_test = train_test_split( X, y,
14                                                    test_size=0.2, random_state=88)
15 # Train random forest
16 model = RandomForestClassifier(n_estimators=100, random_state=42)
17 model.fit(X_train, y_train)
```



Random Forest (4)

```
1  # Make predictions
2  y_pred = model.predict(X_test)
3
4  # Evaluation
5  cm = confusion_matrix(y_test, y_pred, labels=[4,2])
6
7  # Plot with clear labels
8  cmp = ConfusionMatrixDisplay(confusion_matrix=cm,
9                               display_labels=['Malignant','Benign'])
10 cmp.plot(cmap='Blues')
11 # Move the x-axis labels to the top
12 plt.gca().xaxis.tick_top()           # move tick labels
13 plt.gca().xaxis.set_label_position('top') # move axis label
14 plt.xlabel('Predicted label')         # reset label text
15 plt.ylabel('True label')              # keep true labels on side
16 plt.show()
17
18 print('\nClassification report:\n',classification_report(y_test,y_pred,
19                                                            target_names=['Malignant','Benign']))
```



Brainstorming 🤔

- 1 Which variable is the most important in classifying human cells as benign or malignant?
- 2 Which one is the least important?

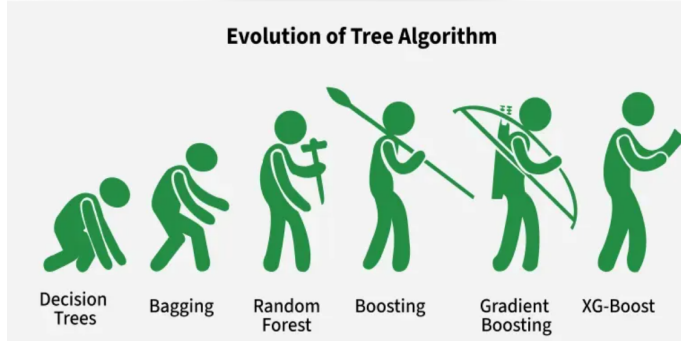


Brainstorming 🤔

- 1 Which variable is the most important in classifying human cells as benign or malignant?
- 2 Which one is the least important?

```
1  # Feature importance
2  importances = model.feature_importances_
3  sorted_features = sorted(zip(X.columns, importances),
4                           key=lambda x: x[1], reverse=True)
5
6  # Put into a table
7  for feature, importance in sorted_features:
8      print(f"{feature}: {importance:.3f}")
```





Source: [Geeks for Geeks](#), 2025

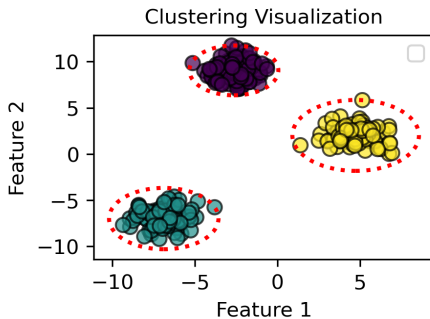
Unsupervised ML algorithms

Clustering

- Used to group similar data points based on their features without using predefined labels

Applications

- Customer segmentation
- Document or text grouping
- Anomaly or outlier detection
- $\vdots$



k-Means (1)

- Groups data into *k*-clusters based on similarity

Steps

- ① Choose the number of clusters $\rightarrow k$
- ② Randomly initialize *k* centroids $\rightarrow$ cluster centres
- ③ Assign each data point to the nearest centroid $\rightarrow$ usually using Euclidean distance

$$d(x_i, \mu_j) = \sqrt{(x_i - \mu_j)^2}$$

- ④ Update the centroids $\rightarrow$ after assignment, each cluster's centroid is recalculated as the mean of all points in that cluster

$$\mu_j = \frac{1}{N_j} \sum_{x_i \in C_j} x_i$$

- ⑤ Repeat 3–4 steps until centroids stop changing significantly $\rightarrow$ convergence



Example

- Consider mall customer dataset: `mall_customers.csv`
- Cluster customers based on their *annual income* and *spending score* (value reflecting the purchasing behaviour of customers)
- Customer segmentation → practice of partitioning a customer base into groups of individuals that have similar characteristics
- Let's do it! 😊



```
1 import numpy np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 import seaborn as sn
5 from sklearn.cluster import KMeans
6
7 df = pd.read_csv('C:/Users/Boss/OneDrive - Universidade de Coimbra/Ambiente d
8 df.head()
9
10 # check for null or missing values
11 df.isna().sum()
12
13 # distribution. any meaningful pattern? we need clustering tho!
14 plt.scatter(df['annual_income'],df['spending_score'])
15 plt.xlabel('Annual income')
16 plt.ylabel('Spending score')
17 plt.show()
```



```
1  # create and fit k-means with 3 clusters
2  X = df[['annual_income', 'spending_score']]
3  kmeans = KMeans(n_clusters=3, random_state=55)
4  kmeans.fit(X)
5  y_kmeans = kmeans.predict(X)
6
7  # print cluster labels and centroids
8  print('Cluster labels:', kmeans.labels_)
9  print('Centroids:', kmeans.cluster_centers_)
```

```
1 labels_sample = kmeans.labels_  
2 df['label'] = labels_sample  
3 sns.scatterplot(x='annual_income', y='spending_score',  
4                 hue='label', palette='Set1', data=df)  
5 plt.scatter(centers[:, 0], centers[:, 1], c='black', s=200,  
6             alpha=0.75, marker='X')  
7 plt.xlabel('Annual income')  
8 plt.ylabel('Spending score')  
9 plt.title('k-Means Clustering Result')  
10 plt.show()  
11
```

Interpretation

- Each color in the plot represents one cluster assigned by k -means
- Red X marks show the cluster centroids (average position of points in each cluster)



Optimum k ?

- No predefined statistical methods to find the optimum value of k
- So, how do we choose it?

Hint

- ① Elbow method
- ② Grid search
- ③ k -fold cross-validation
- ⋮

Reading assignment!



Before fitting any ML model:

- 1 Stay calm and grab your coffee ☕
- 2 Clearly define and understand the business problem – think & rethink 🤔
- 3 Take time on data cleaning and exploration
- 4 Don't open champagne before checking the variance-bias tradeoff. Keep an eye on **overfitting** and **underfitting** problems
- 5 Don't overlook the core concepts of statistics – are you satisfied yourself and able to explain to the boss clearly?
- 6 If 'yes', celebrate! 🎉 Otherwise, go back to step 1 and take another cup of coffee ☕

Thanks. 🙏

Struggling with fitting simple and advanced statistical models? What about deploying machine learning algorithms? Wait ... which programming language do you use? Python? R? SAS? No worries, feel free to reach out to get quality service! 🔥

Contact

✉ tashe.zgreat@gmail.com ☎ +251 92 234 5086

in [LinkedIn](#) GitHub  [Google Scholar](#)

Motto → Quality over quantity!

